

SIMATS ENGINEERING

ASSESSMENT TOOL 2

COURSE NAME: MLA03

COURSE CODE: Reinforcement learning for Environment and Agent

COURSE OUTCOME COVERED

CO2: *Apply Dynamic Programming methods to solve reinforcement learning problems and derive optimal policies for decision-making tasks. (BL2, BL3)*

Assessment Tool Weightage: 30%

Dr G.A. SENTHIL

QUESTIONS: 10

Total Marks: 50

Annexure A

Application - Based Questions

Duration: 2Hrs

Code of Conduct:

I Vignesh S / 192325002 (Name / Reg No) certify that this submission is my original work and that I have adhered to the guidelines specified for this assessment.

I understand that any violation of academic integrity rules will result in disciplinary action.

Signature of the student: Vignesh S

ANSWERS

Q1) RL Framework for Ride-Sharing Driver-Passenger Matching

Objective: Match drivers to passenger requests so as to minimize passenger waiting time and travel distance and improve overall service quality.

Why RL is suitable: Matching is a continuous stream of decisions where assigning one driver now changes which drivers are free for later requests. RL optimizes cumulative service quality across many requests rather than each request in isolation.

Agent and environment: The agent is the matching and dispatch engine. The environment is the city with its drivers, waiting passengers, road traffic and demand.

State space: Driver locations and availability, pending passenger requests and pickup points, current traffic conditions, and demand-supply balance across zones.

Action space: Assign a specific driver to a request, reject or defer a request, or reassign a nearby driver to a higher-priority pickup.

Reward function: Positive reward for short passenger wait, short pickup distance and high rider satisfaction. Negative reward for long waits, cancellations and idle driver time. This shaping directly encourages reduced waiting time and better service quality.

Policy and action selection: A learned policy scores candidate drivers for each request and selects the best match, mostly exploiting learned values while exploring alternative assignments to keep improving.

Learning and continuous improvement: The agent learns from the outcome of each match, such as realized wait time and whether the trip completed. Over many interactions it improves its matching decisions, adapting to rush hours, events and shifting demand across the city.

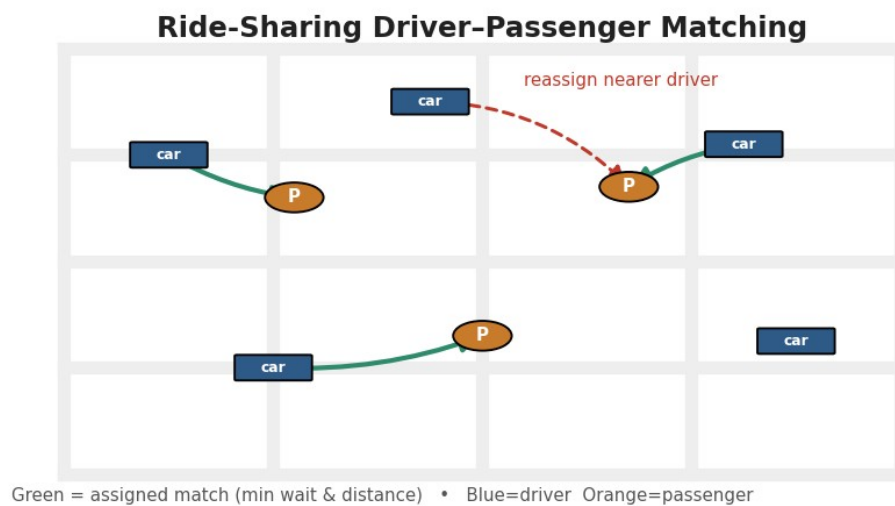


Figure 1: RL Framework for Ride-Sharing Driver-Passenger Matching.

Expected outcomes: Reduced passenger waiting time, efficient driver utilization, fewer cancellations, higher service quality and increased customer satisfaction.

Q2) RL for Personalized Study-Material Recommendation in E-Learning

Objective: Recommend study materials that maximize each learner's understanding and progress.

Why RL is suitable: Learning is sequential; the right next material depends on what the learner has already mastered, and RL optimizes long-term learning gain across a course.

Agent and environment: The agent is the recommender; the environment is the learner interacting with a library of lessons, quizzes and exercises.

State space: Learner's current mastery per topic, past performance and attempts, learning goals, and engagement level.

Action space: Recommend a familiar review item, a new topic, an easier or harder exercise, or a different content format.

Reward function: Positive reward for improved quiz scores, mastery gains and sustained engagement. Negative reward for drop-off, repeated failure and disengagement.

Exploration versus exploitation: This is the heart of the problem. Exploitation recommends familiar content the learner already does well on, which feels comfortable but adds little new learning. Exploration introduces new or harder material that may cause short-term struggle but drives real progress. Balancing the two, for example with an epsilon-greedy or bandit strategy, keeps learners both challenged and confident.

Learning and continuous improvement: The agent updates its policy from each learner's responses, personalizing recommendations and improving as more interaction data accumulates.

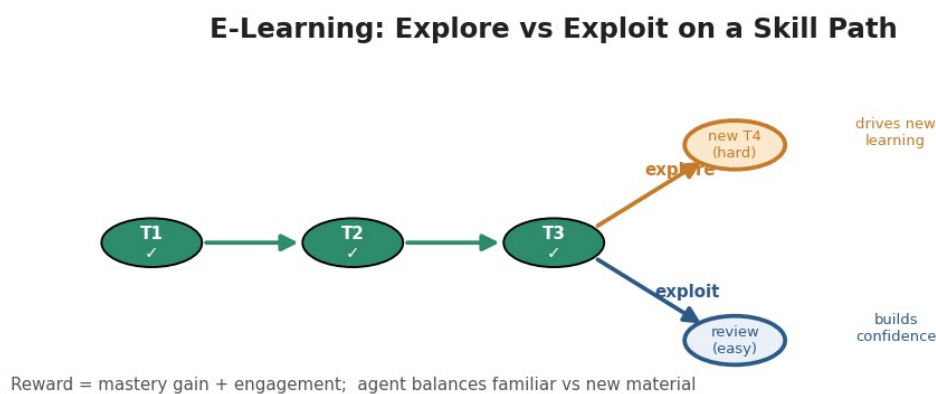


Figure 2: RL for Personalized Study-Material Recommendation in E-Learning.

Expected outcomes: Faster mastery, higher engagement and completion, and personalized paths that adapt to each learner.

Q3) RL Framework for a Robotic Vacuum Cleaner

Objective: Clean a floor area efficiently and completely while avoiding obstacles and managing battery.

Why RL is suitable: Cleaning is a sequential navigation task in an unknown, changing layout, so a learned policy adapts better than fixed movement patterns.

Agent and environment: The agent is the vacuum's controller; the environment is the home floor plan with furniture, dirt and a charging dock.

State space: Robot position and orientation, the map built so far, sensed dirt, battery level, and nearby obstacles.

Action space: Move forward or back, turn left or right, and return to the dock to recharge.

Reward function: Positive reward for newly cleaned area and for docking before the battery dies. Negative reward for collisions, revisiting clean areas and getting stuck.

Concept of policy: The policy maps the robot's current state to a movement action. Early on it is close to random, but as learning proceeds the policy becomes a systematic coverage strategy such as following walls and filling in gaps.

How the value function improves performance over time: The value function estimates the expected future cleaning reward from each state. As the robot gathers experience, these estimates sharpen, so the policy derived from them chooses movements that cover more area with less backtracking and battery use, and performance improves each run.

Robotic Vacuum: Coverage Path & Obstacles



Figure 3: RL Framework for a Robotic Vacuum Cleaner.

Expected outcomes: More complete coverage, fewer collisions, better battery efficiency, and cleaning behaviour that improves with use.

Q4) RL for a Cryptocurrency Trading System

Objective: Decide when to buy, sell or hold a cryptocurrency to maximize risk-adjusted profit.

Why RL is suitable: Trading is sequential and the market is highly volatile and non-stationary; RL can learn adaptive strategies that optimize long-term return.

Agent and environment: The agent is the trading bot; the environment is the crypto market with its prices, volatility and order book.

State space: Current and recent prices, volatility, trading volume, current holdings and cash, and technical indicators.

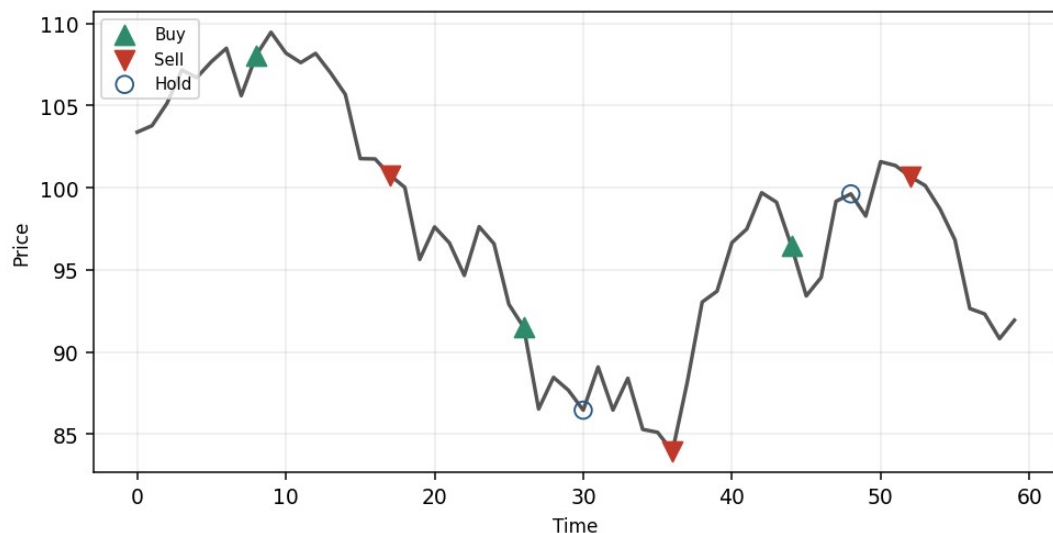
Action space: Buy, sell or hold, optionally with a chosen position size.

Reward function and its design challenges: A natural reward is realized profit, but designing it well is difficult. Rewarding raw profit alone pushes the agent toward high-risk, high-variance bets; ignoring transaction fees leads to over-trading; and a purely short-term reward encourages chasing noise. Better designs use risk-adjusted returns, penalize drawdowns and fees, and discount appropriately.

How poor reward design affects performance: If the reward is mis-specified, the agent optimizes the wrong thing: it may look profitable in backtests but take ruinous risks, over-trade away its gains in fees, or over-fit to past volatility and fail live. Reward design therefore determines whether the learned policy is genuinely sound or dangerously fragile.

Policy, learning and improvement: A policy-gradient or value-based method learns from historical and simulated price paths, refining the policy as new market data arrives, always within strict risk limits.

Cryptocurrency Trading: Buy / Sell / Hold Signals



Reward must be risk-adjusted: raw-profit rewards encourage risky over-trading

Figure 4: RL for a Cryptocurrency Trading System.

Expected outcomes: Potentially better risk-adjusted returns and adaptivity, but only with a carefully designed reward and firm risk controls.

Q5) RL Model for Smart-Grid Electricity Distribution

Objective: Distribute electricity to balance supply and demand, reduce peak load and keep the grid reliable.

Why RL is suitable: Distribution decisions are time-coupled through storage and demand, and conditions are uncertain, so a learned adaptive policy outperforms fixed schedules.

Agent and environment: The agent is the grid distribution controller; the environment is the grid with its generators, storage, consumers and time-varying demand.

State space: Current demand, available generation and renewable supply, storage level, time of day, and electricity price.

Action space: Route power to regions, charge or discharge storage, defer flexible load, or shed non-critical load.

Reward mechanism: Positive reward for meeting demand reliably and using cheap or renewable supply. Negative reward for peak-hour consumption, unmet demand and outages.

Policy and action selection: A learned policy schedules distribution and storage, exploiting good strategies while exploring under unusual demand or supply.

Learning and improvement: The agent learns demand and supply patterns from historical and live data, improving its balancing decisions and flattening peaks over time.

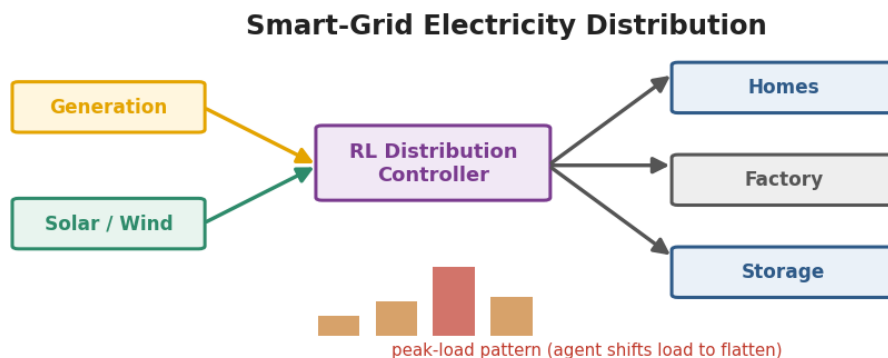


Figure 5: RL Model for Smart-Grid Electricity Distribution.

Expected outcomes: Reduced peak load, higher reliability, lower cost and better integration of renewable energy.

Q6) RL for a Healthcare Medication-Dosing System

Objective: Adjust medication dosage over time to keep a patient in a healthy target range while avoiding adverse effects.

Why RL is suitable: Dosing is a sequence of decisions where each dose affects the patient's future state, and responses vary between patients, so RL can personalize dosing from observed response.

Agent and environment: The agent is the dosing advisor; the environment is the patient's physiology, monitored through vitals and lab tests.

States: Vital signs, relevant lab values, medical history, current dosage, and observed response so far.

Actions: Increase, decrease or maintain the dose, or pause treatment.

Rewards: Positive reward for moving the patient toward and holding the target health range. Strong negative reward for adverse events, toxicity or overdose.

Policy and action selection: A conservative learned policy recommends dose changes, exploring only within clinically safe bounds and always under clinician supervision.

How learning improves outcomes: As the agent observes how patients respond to doses, it refines its value estimates and personalizes recommendations, so dosing becomes more accurate and safer over time. Delayed and noisy responses and patient variability mean the reward must be handled carefully and safety constraints enforced.

Medication Dosing: Response Toward Target Range

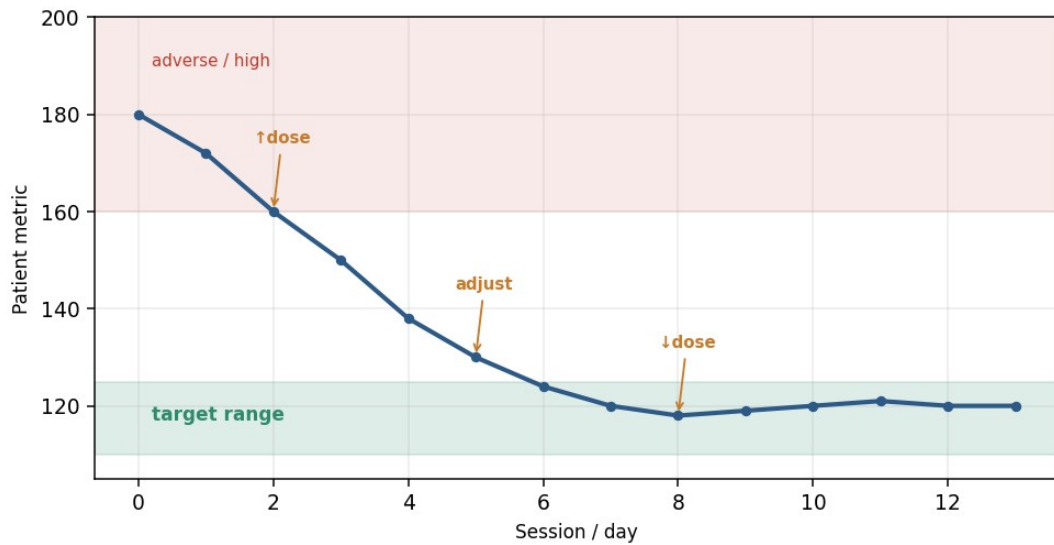


Figure 6: RL for a Healthcare Medication-Dosing System.

Expected outcomes: More personalized, stable dosing, fewer adverse events, and treatment that adapts to each patient, with human oversight throughout.

Q7) RL Model for a Drone Delivery Navigation System

Objective: Navigate a delivery drone from origin to destination quickly and safely while avoiding obstacles and no-fly zones.

Why RL is suitable: Navigation is sequential and the airspace is dynamic with wind, obstacles and battery limits, so a learned policy adapts better than fixed flight plans.

Agent and environment: The agent is the drone's navigation controller; the environment is the airspace with wind, obstacles, no-fly zones and delivery points.

State space: Drone position and velocity, wind conditions, battery level, sensed obstacles, and distance to the destination.

Action space: Change heading, adjust altitude, change speed, or re-route around a hazard.

Reward function: Positive reward for progress toward and successful delivery at the destination. Negative reward for collisions, entering no-fly zones, excessive battery use and long detours.

Exploration and exploitation in route optimization: Exploration lets the drone try alternative routes and altitudes to discover faster or safer paths, while exploitation flies the best-known route. Early in training exploration dominates so the agent maps out good corridors; later a small exploration rate lets it adapt to new wind or obstacles without abandoning proven routes. The balance directly shapes how well routes are optimized.

Learning and improvement: The agent trains largely in simulation and refines its policy from flight data, improving route efficiency and safety over time.

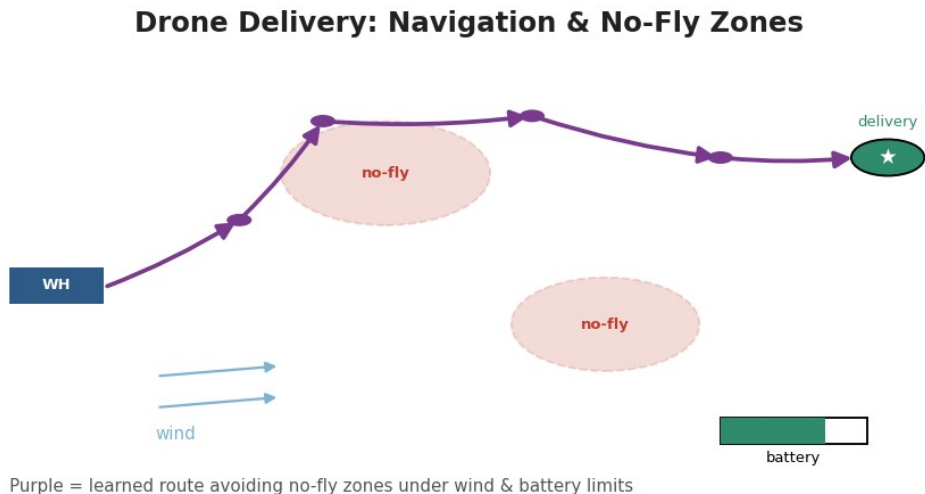


Figure 7: RL Model for a Drone Delivery Navigation System.

Expected outcomes: Faster and safer deliveries, better battery efficiency, reliable obstacle and no-fly-zone avoidance, and routes that keep improving.

Q8) RL Framework for a Manufacturing Assembly Robot

Objective: Optimize a robot's assembly operations to raise throughput and quality while reducing cycle time and defects.

Why RL is suitable: Assembly is a sequence of coordinated actions where order and timing matter, and RL can learn efficient, adaptable operation sequences from experience.

Agent and environment: The agent is the robot's control policy; the environment is the assembly line with parts, tools and workpieces.

State space: Positions and orientations of parts, gripper and tool state, current stage of assembly, and measured cycle time.

Action space: Pick, place, weld or fasten a part, adjust grip or force, or reorder the operation sequence.

Reward function: Positive reward for correctly completed assemblies and faster cycles. Negative reward for defects, misalignment, rework and collisions.

Concept of policy: The policy maps the current assembly state to the next best action, encoding a learned operating procedure rather than a hand-coded script, which lets it handle variation in parts and placement.

How value functions improve efficiency: The value function estimates the expected future reward, essentially the quality and speed of finishing the assembly, from each state. As the robot gains experience these estimates improve, so the policy chooses actions that reduce cycle time and defects, and efficiency rises over successive runs.

Assembly Robot: Operation Sequence & Cycle

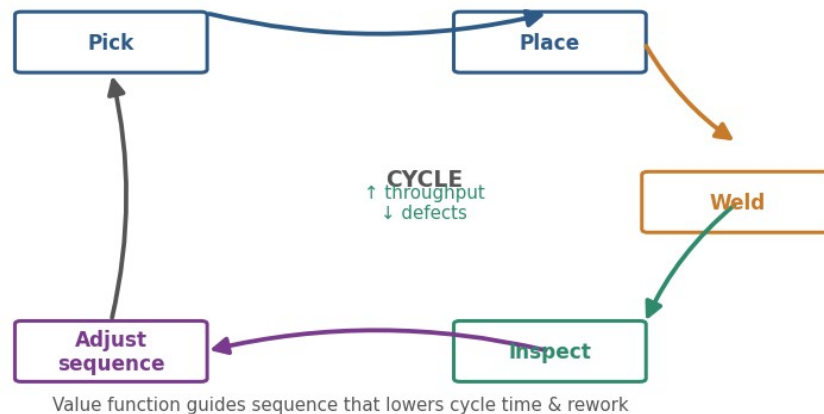


Figure 8: RL Framework for a Manufacturing Assembly Robot.

Expected outcomes: Higher throughput and yield, fewer defects and less rework, and assembly that adapts to part variation and improves over time.

Q9) RL for an Online Advertising System (Maximizing CTR)

Objective: Choose which advertisements to show so as to maximize click-through and conversions over time.

Why RL is suitable: Ad selection is a repeated decision under uncertainty about user interest, and RL optimizes long-term click and conversion value while learning user preferences online.

Agent and environment: The agent is the ad-serving engine; the environment is the stream of users and page contexts together with the available ad inventory.

State space: User profile and history, page and session context, and features of candidate ads.

Action space: Select which ad to display, or how much to bid for an impression in an auction.

Reward function and its design challenges: The obvious reward is a click, but designing the reward is hard. Clicks are sparse and noisy, optimizing only for clicks encourages clickbait that hurts long-term trust, and true value is a later conversion or purchase that arrives with delay. A good reward blends clicks with conversions, discounts for delay and penalizes wasted spend.

Impact of reward design on performance: If the reward rewards clicks alone, the system learns to chase sensational but low-value ads, conversions and user satisfaction fall, and revenue suffers even as click counts look high. Aligning the reward with genuine business value is therefore decisive for real performance.

Policy, learning and improvement: A contextual-bandit policy selects ads, exploring uncertain ads while exploiting proven ones, and updates continuously as clicks and conversions are observed.

Online Advertising: Selection Funnel (CTR)

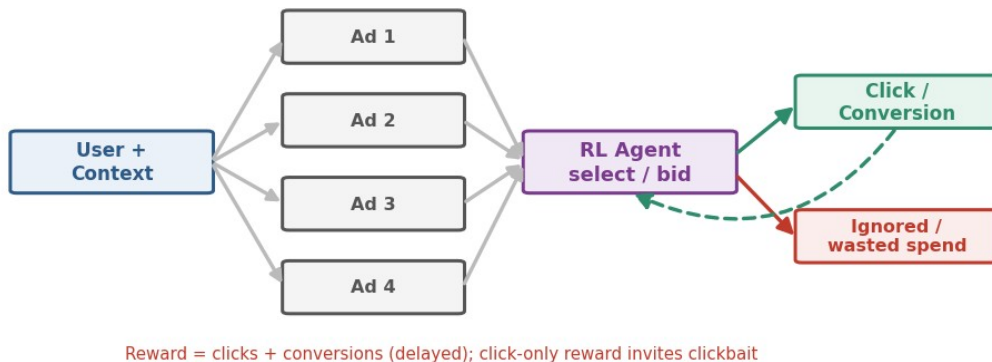


Figure 9: RL for an Online Advertising System (Maximizing CTR).

Expected outcomes: Higher effective click-through and conversion, better use of ad budget, and steadily improving targeting when the reward is well designed.

Q10) RL Framework for a Smart Irrigation System

Objective: Optimize water usage so crops stay healthy while wasting as little water as possible.

Why RL is suitable: Irrigation decisions play out over a season, their effect on crops is delayed, and weather is uncertain, so a learned adaptive policy suits the problem well.

Agent and environment: The agent is the irrigation controller; the environment is the field with its soil, crops and weather.

State space: Soil moisture, weather forecast such as rain and temperature, crop growth stage, and time since the last watering.

Action space: Irrigate a chosen amount, schedule watering for later, or stay off.

Reward function: Positive reward for healthy crop growth and good yield. Negative reward for water waste, over-watering and crop stress from under-watering.

Policy and action selection: A learned policy decides watering amounts from the current state, exploiting good schedules while exploring under unusual weather.

Learning and improvement: Because the payoff appears later as crop health and yield, the agent uses discounting to credit earlier watering. It learns from soil and weather data across the season and improves its schedule over successive cycles.

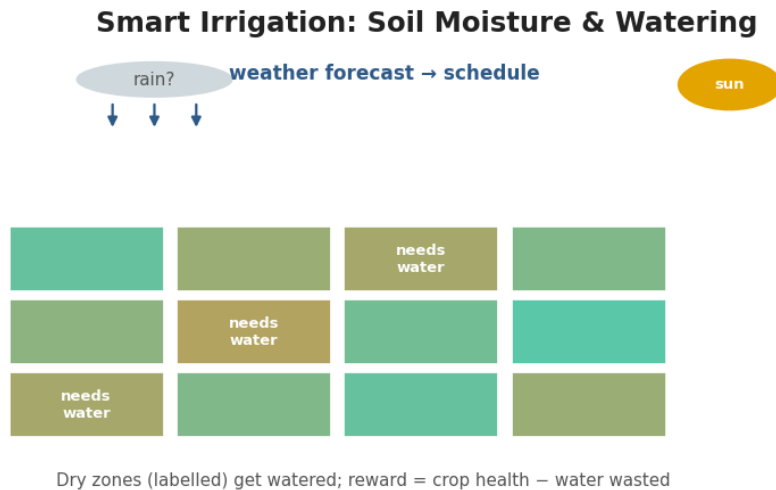


Figure 10: RL Framework for a Smart Irrigation System.

Expected outcomes: Lower water consumption, healthier crops and better yield, and irrigation that adapts to weather and soil conditions.

RUBRICS FOR CALCULATION

Criteria	Weightage	Level 4 – Exemplary	Level 3 – Proficient	Level 2 – Developing	Level 1 – Beginning
Understanding of Problem Context	20	Clearly understands the problem and accurately identifies all requirements	Good understanding with minor gaps	Basic understanding; some requirements unclear	Poor understanding or misinterpretation of the problem
Application of Concepts	30	Accurately applies appropriate concepts to solve complex problems	Applies relevant concepts correctly with minor errors	Applies basic concepts with limited accuracy	Fails to apply appropriate concepts
Problem-Solving Approach	20	Logical, well-structured, and efficient approach	Mostly logical approach with minor gaps	Basic approach with some inconsistencies	Illogical or unclear approach
Accuracy of Solution	20	Completely correct solution with proper justification	Mostly correct with minor errors	Partially correct solution	Incorrect or incomplete solution
Clarity	10	Clear, well-organized, and properly explained	Understandable with minor clarity	Limited clarity and explanation	Poorly presented and difficult to understand

		solution	issues		
--	--	----------	--------	--	--